Understanding Number Representation Systems

Decimal System (Base-10)

- Most familiar number system, composed of digits 0 to 9.
- Numbers are represented as a sum of powers of 10.

- **Example**:

147.3 in base-10 is expanded as:

1 x 10^2 + 4 x 10^1 + 7 x 10^0 + 3 x 10^{-1}

Base-N Systems

- Any number system can use a base N , where N is the number of unique digits in the system.
- Example (Base-3):

121 (base-3) = 1 x 3^2 + 2 x 3^1 + 1 x 3^0 = 16 (base-10)

Binary Numbers (Base-2)

- A key number system for computers, using only digits 0 and 1.
- Each digit is a coefficient of a power of 2, also known as a bit.

Conversion Examples

- **11 (Base-10)** in binary:

11 (base-10) = 1011 (base-2)

- **37 (Base-10)** in binary:

37 (base-10) = 100101 (base-2)

Binary Arithmetic

Binary Addition and Multiplication

- **Addition Example:**

Adding 37 and 17 in binary:

100101 + 10001 = 110110 , which is 54 in decimal.

- **Multiplication Example:**

Multiplying 37 by 17 in binary:

100101 x 10001 = 1001110101 , which is 629 in decimal.

Why Binary is Useful in Computers

•Operations on binary numbers are implemented using logic gates (AND, OR, NOT), which are fast for computers to execute.

Limitations

•Binary numbers are stored in fixed bit lengths (e.g., 32-bit systems), restricting the range of representable numbers.

Floating Point Numbers

What are Floating Point Numbers?

- Floating point numbers are used to represent real numbers in a computer, providing a wide range of values while using a fixed number of bits.
- They are crucial for scientific and engineering calculations due to the need for high precision and a broad range of values.

Components of Floating Point Numbers:

- **Sign (s):** Determines whether the number is positive or negative.
- **Exponent (e):** The power of 2, indicating the scale of the number.
- **Fraction (f):** The significant or mantissa, a value between 1 and 2, indicating the precision of the number.

IEEE 754 Double Precision Format:

- Python maps floats to the IEEE754 double precision format, which uses **64 bits**:
  - **1 bit for the sign**, **11 bits for the exponent**, and **52 bits for the fraction**.
- The exponent has a bias of 1023, meaning e - 1023 normalizes the exponent.

Calculations and Precision

Representation of a Floating Point Number:

- A float can be represented as:

n = (-1)^s x 2^(e-1023) x (1 + f)

Example:

- For the binary number:

1 10000000010 1000000000000000000000000000000000000000000000000000

- **Sign** s = 1 (negative)
- **Exponent** e = 1026 (decimal: 3)
- **Fraction** f = 1.5

The exponent in decimal is $1 \cdot 2^{10} + 1 \cdot 2^{1} - 1023 = 3$. The fraction is $1 \cdot \frac{1}{2^1} + 0 \cdot \frac{1}{2^2} + \ldots = 0.5$. Therefore $n = (-1)^1 \cdot 2^3 \cdot (1 + 0.5) = -12.0$ (base10). See the following figure for details.

Precision Considerations:

•The gap between two adjacent floating point numbers increases with the magnitude of the numbers.

•The function numpy.spacing(x) can be used to find the distance (gap) between numbers, which grows as x increases.

Special Cases and Limitations

Special Floating Point Cases:

- **Subnormal Numbers:** Occur when the exponent is zero, allowing representation of very small numbers.
- **Infinity and NaN (Not a Number):** Occur when the exponent is all 1s (2047). If the fraction is zero, the result is infinity; if not, the result is NaN.

Handling Overflow and Underflow:

- **Overflow:** Occurs when a result exceeds the largest representable number, handled by Python as inf.
- **Underflow:** Occurs when a result is smaller than the smallest representable subnormal number, handled by Python as 0.

Understanding Round-Off Errors in Numerical Computations

What are Round-Off Errors?

- Round-off error occurs due to the approximation of numbers when stored in a computer's memory.
- These errors happen because computers represent floating-point numbers in binary, which cannot always perfectly represent decimal numbers.
- **Key Concept:** The difference between the actual value of a number and its approximation in the system is called a round-off error.

Representation Error:

- Common cause of round-off errors.
- Example: pi (3.14159…) or 1./3. (0.33333…) can't be perfectly represented, leading to small but cumulative errors in calculations.
- Rounding Multiple Times: Successive rounding can exacerbate errors. For instance:
  Rounding 4.845 to two decimals: 4.85, then to one decimal: 4.9, leading to a larger cumulative error than rounding directly to one decimal: 4.8.

Examples of Round-Off Errors in Python

Floating Point Arithmetic Issues

- Operations in Python demonstrate how floating-point numbers accumulate round-off errors.
- Example:
  - 4.9 - 4.845 in Python yields 0.05500000000000604, not exactly 0.055.
  - 0.1 + 0.2 + 0.3 != 0.6 due to rounding approximations.

Accumulation of Round-off Error:

- Errors accumulate over repetitive calculations.
- Example: Adding and subtracting 1/3 multiple times:
  - If done once: 1 + 1/3 - 1/3 evaluates to 1
  - Repeated 100 times then evaluate: Result becomes 1.0000000000000002
  - As the number of repetitions increases, the error increases proportionally.